

ENTERPRISE SYSTEM DOCUMENTATION

ASSETCARE

IT Asset Management & Asset Care Lifecycle Management System

Comprehensive Technical Architecture, Multi-Tenant Data Flow, Commercial Subscription & Security Specification.

PREPARED FOR

Executive Management & Technical Leadership

CURRENT BUILD VERSION

v1.0.0-PROD (Vite 8 / Node 22 / React 19)

PLATFORM ARCHITECTURE

Multi-Tenant SaaS / Cloud & Standalone Windows Intranet

DOCUMENT TYPE

System Architecture & Technical Documentation

DOCUMENT DATE

September 2026

SECURITY & COMPLIANCE

Isolated Multi-Tenancy, HMAC SHA256 Webhooks, RBAC

Table of Contents

1. Executive Summary	3	2. Project Overview	4
3. Business Purpose & Workflow	4	4. High-Level System Architecture	5
5. Technology Stack & Versions	6	6. Project & Codebase Structure	7
7. Frontend Architecture & Routing	8	8. Backend Architecture & Pipeline	9
9. Database Architecture & Models	10	10. Multi-Tenant Isolation Model	11
11. User Roles & Permission Matrix	12	12. Authentication & Session Engine	13
13. Core AssetCare Modules	14	14. Subscription Plans & Pricing	16
15. Plan Entitlement Enforcement	16	16. 365-Day Subscription Lifecycle	17
17. Checkout Architecture	18	18. Upgrade, Renewal & Cancellation	18
19. Downgrade Policy Specification	19	20. Coupon & Discount Engine	19
21. GST & Billing Mathematics	20	22. Razorpay Gateway Architecture	21
23. Invoice Generation Pipeline	22	24. Super Admin Platform Console	23
25. Company Deactivation Protocol	24	26. API Architecture & Endpoints	25
27. Security & Defense Architecture	27	28. Complete Business Workflows	28
29. Build & Compilation Pipeline	29	30. Desktop Installer Architecture	30
31. Project Footprint & Top Files	31	32. Complete Dependency Inventory	32
33. Environment Variables Reference	33	34. Quality Assurance & Test History	34
35. Data Integrity & Segregation	35	36. Current Database Production State	35
37. Technical Limitations	36	38. Technical Debt Assessment	36
39. Risk Analysis Matrix	37	40. Overall System Status Matrix	38
41. Management Summary	39	42. Architectural Conclusion	40
App. Appendices A – E	41		

1. Executive Summary

AssetCare is an enterprise-grade, multi-tenant IT Asset Management (ITAM) and Asset Care Lifecycle Management System. It is engineered to solve critical operational inefficiencies that growing companies and enterprises face when tracking physical hardware, assigning devices to personnel, handling equipment breakdowns, managing vendor service contracts (AMCs), tracking warranties, and maintaining statutory audit compliance.

The platform operates under a dual-delivery model: it functions as a highly scalable multi-tenant Cloud SaaS platform and can also be distributed as a standalone, zero-external-dependency Windows desktop/intranet application. Organizations subscribe to annual commercial tiers (**Home User**, **MSME**, or **Large Scale**), gain access to dedicated isolated workspaces, and onboard their IT teams, department heads, technicians, and employees.

Key Business Capabilities Implemented

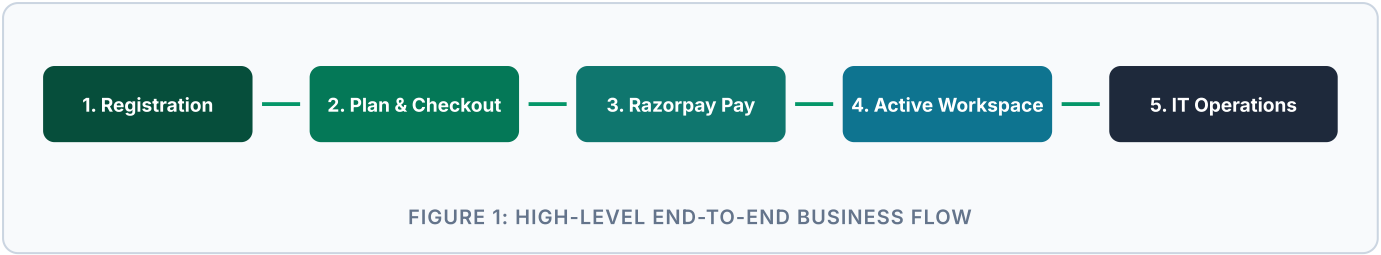
- **End-to-End Asset Lifecycle Tracking:** Procurement, asset tagging (QR code), departmental allocation, custodian assignment, depreciation, and decommission.
- **Integrated Maintenance & Helpdesk:** SLA-governed ticketing, repair workflow escalation, external service center management, and cost tracking.
- **Strict Multi-Tenant Data Enclosure:** Complete logical and cryptographic isolation between tenant databases and collections.
- **Automated Commercial Billing & Payments:** Automated GST invoicing (CGST/SGST/IGST), coupon discount engine, Razorpay checkout integration with HMAC SHA256 webhook reconciliation, and 365-day annual license enforcement.
- **Platform Super Admin Console:** Centralized tenant monitoring, revenue analytics, company activation/deactivation controls, coupon creation, and 360° company data audit.

2. Project Overview

Attribute	Specification / Value
Application Name	AssetCare / IAssetCare (Enterprise ITAM)
Business Model	B2B SaaS Annual Subscription (365-Day Validity) + Enterprise On-Premises
Target Audience	IT Managers, System Administrators, HODs, Technicians, Corporate Employees
Deployment Models	1. Cloud SaaS (Multi-Tenant Web) · 2. Windows Standalone Intranet (Inno Setup / Executable)
Monetization Engine	Tiered Subscriptions (₹999 / ₹2,999 / ₹8,999 / year) via Razorpay Payment Gateway
File Storage Engine	Hybrid Cloudinary CDN (Direct HTTPS) with Automatic Local Disk Fallback
Current Status	CORE FULLY IMPLEMENTED · Production Audited & Hardened

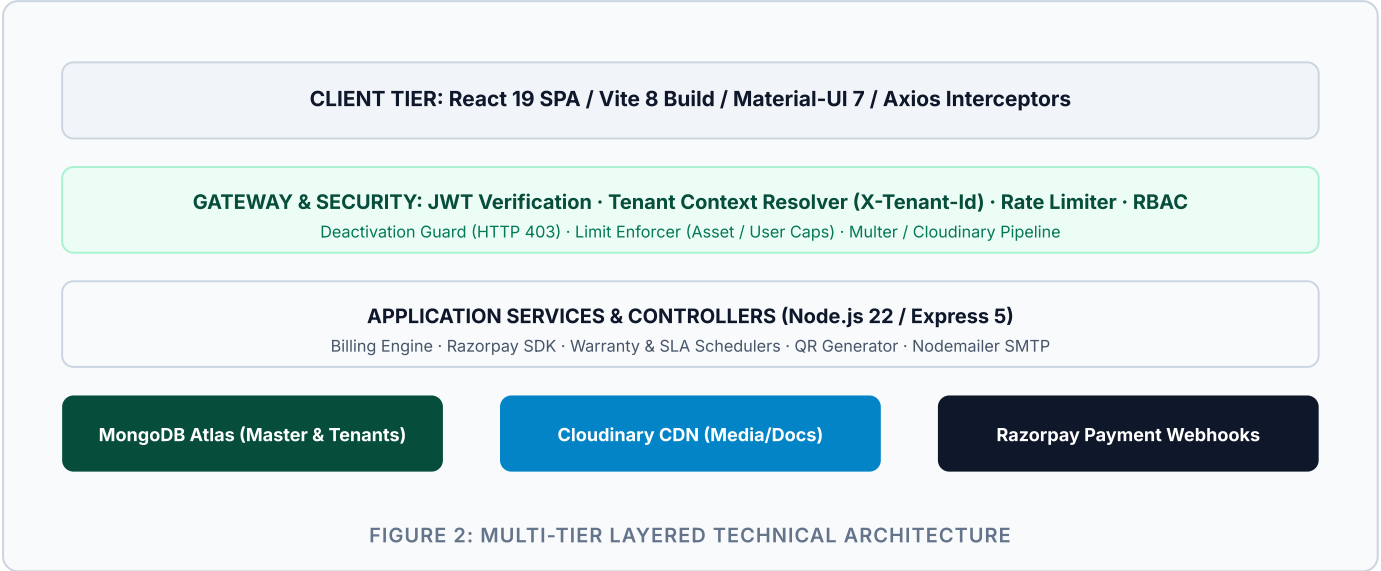
3. Business Purpose & Commercial Workflow

The business workflow bridges organizational equipment tracking with commercial subscription governance:



4. High-Level System Architecture

AssetCare follows a resilient Multi-Tier Client-Server architecture with micro-service modularity in the backend and an asynchronous reactive frontend:



5. Technology Stack

Architecture Layer	Technology / Library	Version	Purpose & Implementation Details
Frontend Framework	React	19.0.0	Single Page Application core, Hooks-based state management
Build System	Vite	8.0.16	ESM bundler, chunk-splitting, fast HMR, production dist builder
UI Component System	Material UI (MUI)	7.3.7	Enterprise design system, ThemeContext, responsive data tables
Routing & Navigation	React Router DOM	7.1.3	Client-side routing, protected auth guards, lazy loaded routes
Charts & Dashboards	Recharts	2.15.1	Asset distribution, maintenance costs, ticket volume analytics
Client Document Export	jsPDF + autoTable	4.2.1	Client-side PDF generation for invoices, data export, QR sheets
HTTP API Client	Axios	1.7.9	REST client with dynamic tenant header and auth interceptors
Backend Runtime	Node.js	22.16.0	Server-side JavaScript runtime engine
Web Framework	Express	5.2.1	RESTful API routing, JSON parser, middleware chaining
Database & ODM	MongoDB Atlas + Mongoose	9.7.1	NoSQL cloud persistence, multi-tenant connection pooling
Authentication	JWT + bcryptjs	9.0.3 / 3.0.3	Stateless JWT token signing, salted SHA-512/bcrypt hashing
Payment Gateway	Razorpay Node SDK	2.9.8	Order generation, checkout invocation, HMAC signature checks
Cloud Media Storage	Cloudinary SDK	2.x	Streamified memory upload to Cloudinary CDN with disk fallback
Task Scheduling	node-cron	4.5.0	Warranty expiration jobs, SLA breach monitors, audit purges
Desktop Installer	Inno Setup / pkg	6.x / 5.8.1	Windows x64 standalone executable bundler

6. Project Structure

```
AssetCare/
├── frontend/                                # React 19 Frontend Client (Vite 8)
│   ├── public/                             # Static assets (favicons, brand logos, sample images)
│   └── src/
│       ├── api/axios.js                   # Configured Axios instance with tenant/auth interceptors
│       ├── components/                   # Shared UI (Navbar, Footer, Sidebar Layout, QR, Modal)
│       ├── context/                      # AuthContext (session state), ThemeContext (dark/light)
│       ├── pages/
│       │   ├── admin/                   # AdminDashboard, Assets, Users, Approvals, Billing, Invoices
│       │   ├── auth/                   # AuthPage (Login), RegisterCompany, Password Reset
│       │   ├── billing/                # Checkout, InvoiceView, Plan Upgrades
│       │   ├── employee/               # EmployeePortal (My assigned assets, raise ticket)
│       │   ├── public/                 # Home, Features, Modules, Workflow, Contact, Terms, Policies
│       │   ├── shared/                 # Tickets, Settings, Notifications, ScanAsset
│       │   └── superadmin/             # SuperAdminPanel (Company oversight, coupons, revenue)
│       └── routes/                     # ProtectedRoute, AdminRoute, SuperAdminRoute guards
│   └── dist/                           # Compiled production assets (6.72 MB total footprint)
├── backend/                             # Node.js 22 & Express 5 API Server
│   ├── config/                          # DB connection pool, planDefaults.js, cloudinary.js
│   ├── controllers/                    # authController, assetController, billingController, etc.
│   ├── jobs/                          # warrantyScheduler.js, slaEscalationJob.js
│   ├── middleware/                     # authMiddleware, tenantMiddleware, limitMiddleware, upload.js
│   ├── models/                        # Tenant, User, Asset, Ticket, Invoice, Coupon, SubscriptionHistory
│   ├── routes/                        # REST route declarations (/api/auth, /api/assets, /api/billing)
│   ├── services/                      # emailService.js (Nodemailer SMTP transactional mail)
│   ├── uploads/                       # Local disk fallback directory (avatars, logos, invoices, docs)
│   ├── server.js                      # Main server entry point, static host, error handler
│   └── release-exe/                   # Compiled Windows setup installers (iassetcare-setup.exe)
```

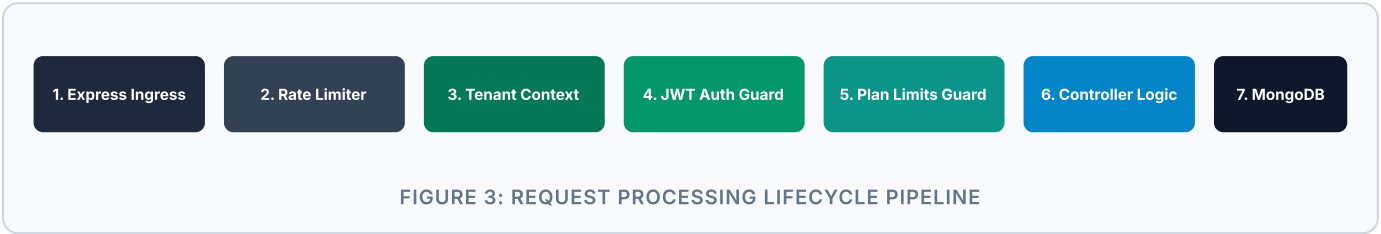
7. Frontend Architecture

The frontend uses React 19 with client-side routing and component-level code splitting (`React.lazy()`). This keeps initial bundle load under **400 KB** while loading heavy features (such as Excel import/export or PDF canvas rendering) strictly on-demand.

Page Component	Route	Access Level	Primary Business Function	Key APIs Consumed
Home	/	Public	Landing page, feature showcase, plan pricing table	Public static / assets
AuthPage	/login	Public	Multi-role login (Employee / Admin / Super Admin)	POST /api/auth/login
RegisterCompany	/register-company	Public	New tenant onboarding & auto-slug assignment	POST /api/auth/register-company
Checkout	/checkout	Auth (Admin)	Plan selection, coupon validation, Razorpay modal	POST /api/billing/create-order
AdminDashboard	/admin/dashboard	Admin / HOD	KPI counters, asset breakdown, maintenance trends	GET /api/dashboard/stats
Assets	/admin/assets	Admin / IT	Asset registry table, filter, CSV bulk import, trash	GET/POST /api/assets
UserManagement	/admin/users	Admin	Employee directory, invite token dispatch, offboard	GET/POST /api/users
Tickets	/tickets	All Roles	Helpdesk repair requests, status progression, notes	GET/POST /api/tickets
SubscriptionBilling	/admin/billing	Admin	365-day license tracker, invoice list, upgrade trigger	GET /api/billing/current
SuperAdminPanel	/super-admin/console	Super Admin	Platform dashboard, tenant freeze, coupon generator	GET /api/super-admin/*

8. Backend Architecture & Pipeline

Incoming HTTP requests pass through a strictly ordered pipeline designed for security, multi-tenancy, rate limiting, and business validation before hitting domain controllers:



9. Database Architecture & Models

AssetCare stores data in MongoDB Atlas with strict multi-tenant schemas. The master database holds global platform entities, while tenant collections are logically scoped by `tenantId` and optionally backed by separate tenant databases via connection switching:

Model Name	Collection Scope	Primary Purpose	Key Indexed Fields & Attributes
Tenant	Global / Master	Company master registry & plan entitlements	<code>slug</code> (unique), <code>plan</code> , <code>subscriptionStatus</code> , <code>isActive</code> , <code>limits</code>
User	Tenant Scoped	Authentication, roles, permissions, department	<code>email</code> (unique), <code>password</code> (bcrypt), <code>role</code> , <code>tenantId</code>
Asset	Tenant Scoped	Hardware inventory records & lifecycle state	<code>assetTag</code> , <code>serialNumber</code> , <code>category</code> , <code>status</code> , <code>assignedTo</code>
Invoice	Tenant & Master	Commercial billing receipts & GST accounting	<code>invoiceNumber</code> , <code>amount</code> , <code>gstDetails</code> , <code>razorpayPaymentId</code> , <code>status</code>
SubscriptionHistory	Tenant & Master	Historical audit trail of all plan transactions	<code>tenantId</code> , <code>plan</code> , <code>amountPaid</code> , <code>startDate</code> , <code>endDate</code> , <code>action</code>
Coupon	Global / Master	Promotional discount configurations	<code>code</code> (unique/uppercase), <code>discountType</code> , <code>discountValue</code> , <code>usedCount</code>
Ticket	Tenant Scoped	Breakdown service requests & SLA tracking	<code>ticketId</code> , <code>asset</code> , <code>raisedBy</code> , <code>status</code> , <code>priority</code> , <code>slaDeadline</code>
MaintenanceSchedule	Tenant Scoped	Preventative maintenance tasks & schedules	<code>asset</code> , <code>taskName</code> , <code>frequency</code> , <code>nextDueDate</code> , <code>status</code>
Department	Tenant Scoped	Organizational hierarchy & cost centers	<code>name</code> , <code>hod</code> , <code>budget</code> , <code>tenantId</code>
AuditLog	Tenant Scoped	Immutable compliance trail of all user actions	<code>actor</code> , <code>action</code> , <code>entity</code> , <code>entityId</code> , <code>changes</code> , <code>createdAt</code>

10. Multi-Tenant Architecture & Isolation

Tenant isolation is enforced in every layer of the software stack:

- **Identification:** When a user logs in, their JWT contains their immutable `tenantId`. For API calls, the frontend automatically transmits the `X-Tenant-Id` header.
- **Mongoose Tenant Plugin:** All tenant-scoped Mongoose queries automatically inject `{ tenantId: req.tenantId }` into `find`, `findOne`, `update`, and `count` filters.
- **Cross-Tenant Leakage Prevention:** Even if a malicious user alters an ID parameter in a URL, the query will return `null` (404) because the record's `tenantId` will not match the authenticated session's tenant token.
- **Super Admin Isolation Bypass:** Platform Super Admins have explicit global scope for administrative oversight while retaining strict tenant auditing.

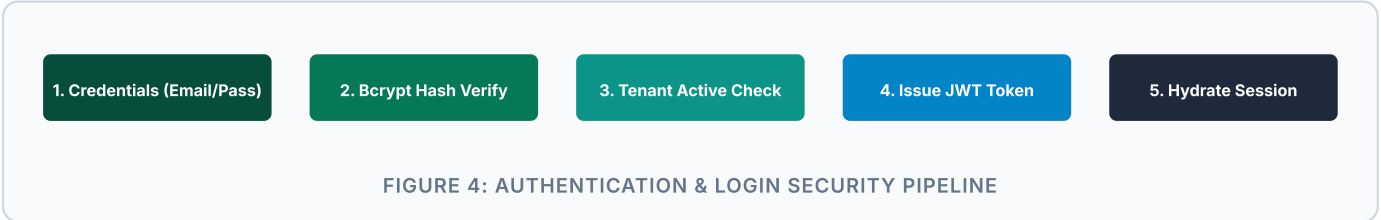
11. User Roles & Permission Matrix

AssetCare implements comprehensive Role-Based Access Control (RBAC) with support for custom granular permissions:

System Capability	Super Admin	Company Admin	Department HOD	Technician	Employee
Platform Company Management	FULL ACCESS	NO ACCESS	NO ACCESS	NO ACCESS	NO ACCESS
Platform Coupon Generation	FULL ACCESS	NO ACCESS	NO ACCESS	NO ACCESS	NO ACCESS
Company Billing & Checkout	VIEW ONLY	FULL ACCESS	NO ACCESS	NO ACCESS	NO ACCESS
Asset Registry & Bulk Import	VIEW ONLY	FULL ACCESS	DEPT SCOPED	VIEW ONLY	NO ACCESS
Asset Assignment & Custody	NO ACCESS	FULL ACCESS	DEPT SCOPED	NO ACCESS	NO ACCESS
Raise Breakdown Ticket	NO ACCESS	FULL ACCESS	FULL ACCESS	FULL ACCESS	ASSIGNED ONLY
Resolve & Update Ticket Status	NO ACCESS	FULL ACCESS	FULL ACCESS	ASSIGNED ONLY	NO ACCESS
User & Employee Onboarding	NO ACCESS	FULL ACCESS	VIEW DEPT	NO ACCESS	NO ACCESS
Custom Branding (Logo Upload)	NO ACCESS	LARGE SCALE	NO ACCESS	NO ACCESS	NO ACCESS
Compliance Audit Log Export	PLATFORM WIDE	COMPANY WIDE	NO ACCESS	NO ACCESS	NO ACCESS

12. Authentication Architecture

Authentication is stateless and cryptographic, utilizing JWT signed tokens:



13. Core AssetCare Modules

Module Name	Target Users	Key Operations & Capabilities	Backend Enforcement / Guard
Asset Registry	Admin, HOD	Register hardware, QR code generation, bulk CSV import, trash/restore	LimitMiddleware checks max asset capacity
Assigned Devices	All Staff	Check equipment custody, serial numbers, allocation history	Employee-scoped queries prevent cross-viewing
Maintenance & AMC	Technicians, Admin	Preventative schedules, AMC vendor contracts, service cost logging	Automated node-cron checks due dates
Tickets & SLA	Employees, Techs	Raise repair requests, technician assignment, SLA breach alarms	Role-checked status transitions
Service Centers	Admin, Techs	Authorized vendor registry, contact directory, repair dispatch	Tenant-scoped CRUD endpoints
User Directory	Company Admin	Invite team members, role assignments, employee offboarding	User limit enforcement per plan tier
Enterprise Workspace	Company Admin	Custom metadata fields, organizational branding, settings	Custom branding restricted to Large Scale
Analytics & Reports	Executive, Admin	Asset depreciation, breakdown rates, spending reports, Excel export	Live aggregation pipeline across collections

14. Subscription Plans & Entitlements

AssetCare provides 3 commercial tiers with strictly bounded resource limits and features:

Commercial Plan	Annual Price	Asset Limit	User Limit	Department Limit	Included Advanced Capabilities
Home User	₹999 / year	Up to 20 Assets	Up to 3 Users	Up to 2 Departments	Core Inventory, Basic Ticketing, Standard Reports
MSME	₹2,999 / year	Up to 50 Assets	Up to 15 Users	Up to 5 Departments	Advanced Analytics, Warranty Tracking, Bulk CSV Import
Large Scale	₹8,999 / year	Unlimited	Unlimited	Unlimited	Custom Branding, SLA Escalation, Dedicated Support, Full Audit Trail

15. Plan Entitlement Architecture

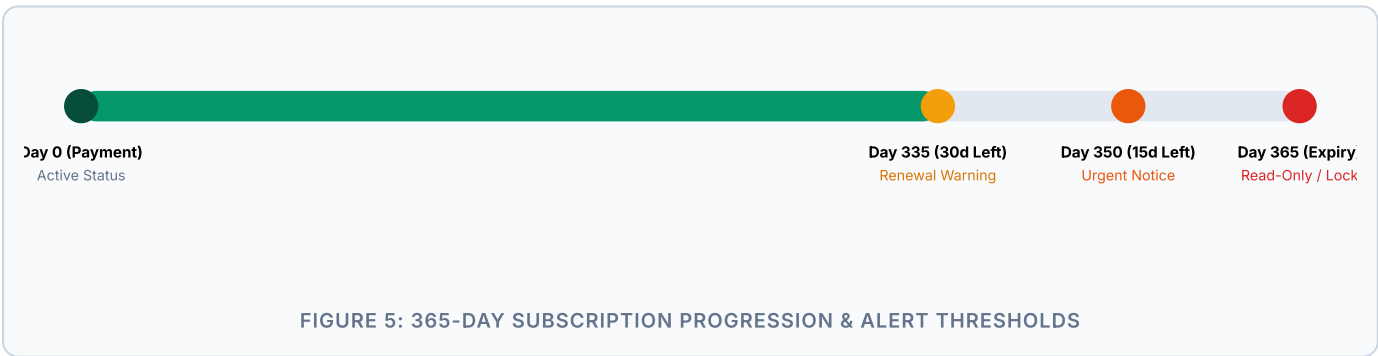
Entitlement boundaries are enforced on both the client UI and the Express backend:

Backend Enforcement Priority

While the frontend disables buttons and hides premium features, the backend middleware (`limitMiddleware.js` and controller feature guards) acts as the primary defense. If a user attempts to bypass the UI and register a 21st asset on the Home User plan via direct API POST, the server returns `HTTP 403 (PLAN_LIMIT_REACHED)` with a descriptive upgrade prompt.

16. 365-Day Subscription Lifecycle

Subscriptions follow a strictly calculated 365-day validity period from the timestamp of payment verification:



17. Checkout, Upgrade & Downgrade Architecture

Upgrade Behavior (Immediate Prorated Transition)

When an organization upgrades (e.g. *Home User* → *MSME* or *MSME* → *Large Scale*), the system processes the upgrade immediately upon payment verification, raises resource caps instantly, and initializes a fresh 365-day term while recording the transition in `SubscriptionHistory`.

Renewal Behavior (Cycle Extension)

Renewing the current active plan extends the subscription for an additional 365 days from the current expiry date without interrupting service or resetting asset registries.

Downgrade Policy

Current System Rule: Immediate Downgrades Are Blocked

Immediate downgrading to a lower plan tier (e.g., Large Scale with 120 assets attempting to downgrade to Home User with a 20-asset limit) is blocked with `HTTP 400 (DOWNGRADE_NOT_ALLOWED)` to prevent immediate data truncation or orphaned assets.

Scheduled Downgrade: Scheduled downgrade at the end of the billing term is not implemented in the current v1.0.0 release.

20. Coupon Engine & Discount Mathematics

Coupons are configured globally by the Super Admin and applied during checkout:

Coupon Attribute	Validation Rule	Impact on Calculation
code	Case-insensitive, converted to uppercase	Matches active record in <code>coupons</code> collection
discountType	'percentage' or 'fixed'	Determines math formula (e.g. 10% off vs ₹500 off)
minOrderAmount	Plan base price must be $\geq$ minOrder	Prevents application on lower tiers if restricted
maxDiscount	Applies as a ceiling cap on percentage discounts	Guarantees discount never exceeds configured rupee cap
validUntil	Must be greater than current UTC date	Rejects expired promotional codes
usageLimit	<code>usedCount</code> must be $<$ <code>usageLimit</code>	Enforces total redemption limits across customers

21. GST & Billing Calculation Engine

All billing calculations are strictly rounded to 2 decimal places (`Math.round(n * 100) / 100`) to eliminate floating-point artifacts:

Mathematical Formula

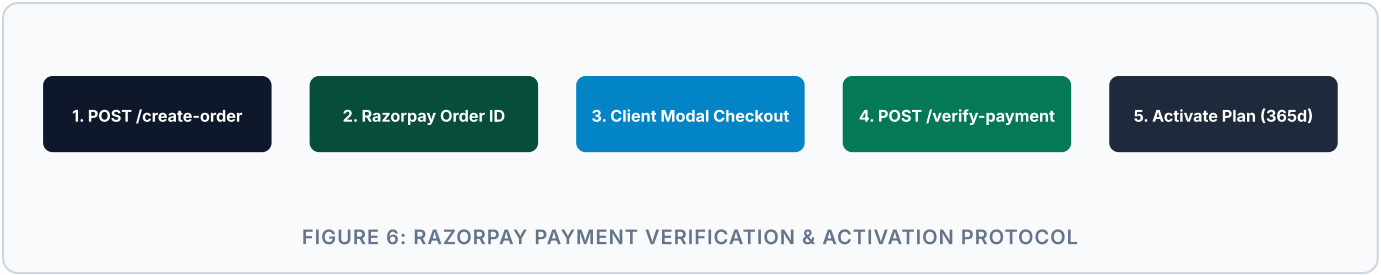
- Base Amount:** Plan Price (P)
- Discount Amount:** $D = \min(\text{CalculatedDiscount}, \text{MaxDiscount})$
- Taxable Base:** $T = P - D$
- GST Computation:**
 - If *Customer State* == *Vendor State (Rajasthan)*: **CGST (9%)** = $T * 0.09$ + **SGST (9%)** = $T * 0.09$
 - If *Customer State* != *Vendor State*: **IGST (18%)** = $T * 0.18$
- Final Grand Total:** $\text{Total} = T + \text{GST}$

Sample Live Calculation Example (MSME Plan with 10% Coupon in Rajasthan)

Base MSME Price	₹2,999.00
10% Promotional Discount	- ₹299.90
Taxable Amount	₹2,699.10
CGST (9%)	+ ₹242.92
SGST (9%)	+ ₹242.92
Final Payable Amount	₹3,184.94

22. Razorpay Payment Architecture

Payment transactions follow a 2-step cryptographically verified order flow:



HMAC SHA256 Signature Verification: The server computes `crypto.createHmac('sha256', secret).update(order_id + '|' + payment_id).digest('hex')`. If the computed signature matches the client's payload, the invoice is set to 'Paid' and the tenant's license is activated.

24. Super Admin Platform Console

The Super Admin Console provides platform owners with complete governance over all tenant instances:

Console Module	Capabilities & Administrative Controls
Platform Analytics	Total registered companies, live subscriber breakdown, gross platform revenue, total GST collected.
Company 360° Management	Full inspection of any tenant's asset counts, user rosters, invoices, and subscription expiration countdowns.
Tenant Freeze / Unfreeze	Immediate administrative deactivation of delinquent or rogue tenants via <code>isActive: false</code> .
Coupon Factory	Create, configure caps, activate, pause, or delete global promotional discount codes.
Contact Leads & Inquiries	Review enterprise inquiries, demo requests, and corporate lead submissions.

25. Company Deactivation & Access Control

When a Super Admin deactivates a company:

1. The master `Tenant.isActive` flag is flipped to `false` in MongoDB.
2. All subsequent API requests with that tenant's JWT are intercepted by `authMiddleware.js`, which detects the deactivated state and immediately returns `HTTP 403 Forbidden (COMPANY_DEACTIVATED)`.
3. The frontend Axios response interceptor captures the 403 response, purges the client local storage session, and redirects the user to `/login?deactivated=1`.

26. Security Architecture

Security Domain	Implementation Mechanism	Threat Mitigated
Authentication	Stateless JWT (HMAC SHA-256) with password bcrypt salting	Credential sniffing, session hijacking
Authorization (RBAC)	Middleware permission guards on all routes (Admin / HOD / Tech / Employee)	Privilege escalation
Data Isolation	Mongoose Tenant Plugin & dynamic tenant connection resolution	Cross-tenant data leakage (IDOR)
Commercial Integrity	Server-authoritative price & GST recalculation on order creation	Client-side price tampering
Payment Security	Server-side HMAC SHA256 verification of Razorpay signatures	Fake payment confirmation exploits
Rate Limiting	<code>express-rate-limit</code> (100 req/15 min in production on auth endpoints)	Brute-force credential attacks & DoS
File Upload Security	MIME-type validation, size caps (2–10 MB), Cloudinary cloud sandboxing	Malicious script execution on server

29. Build, Deployment & Installer Architecture

Production Web Build

- **Vite 8 Compiler:** Compiles React 19 JSX and CSS into `frontend/dist` in ~8.6 seconds.
- **Asset Optimization:** Chunks are hashed and compressed (Gzip/Brotli ready).
- **Static Serving:** In production, Express hosts `frontend/dist` statically and serves `index.html` on SPA route fallbacks.

Windows Standalone Desktop Application

- **Compilation:** Bundled using `pkg` into standalone Node runtime binary `asset-care-backend.exe`.
- **Inno Setup Packaging:** Packaged into a single-file Windows installer: `iassetcare-setup.exe` (374.59 MB).
- **Embedded Services:** Bundles the complete frontend UI, backend API engine, and local database interface.

31. Project Footprint & File Statistics

Project Component	File Count	Size on Disk (MB)	Notes
Frontend Source Code	86 files	5.88 MB	React 19 source, components, pages, styles
Backend Source Code	117 files	1.80 MB	Controllers, models, middleware, jobs, routes
Frontend Production Dist	161 files	6.72 MB	Optimized production JS, CSS, fonts, assets
Frontend Dependencies (node_modules)	64,739 files	280.16 MB	Vite, MUI, React, Recharts, jsPDF, Lucide
Backend Dependencies (node_modules)	6,168 files	31.56 MB	Express, Mongoose, Razorpay, Cloudinary, JWT
Windows Setup Installer (EXE)	1 file	374.59 MB	Standalone Windows offline installer package

Top 10 Largest Codebase Files

Rank	File Path	Size (MB / KB)	Purpose
1	release-exe/iassetcare-setup.exe	374.59 MB	Windows Inno Setup installer package
2	frontend/dist/favicon.png	684.3 KB	High-res application icon asset
3	frontend/dist/logo.png	684.3 KB	Primary brand logo asset
4	frontend/dist/service_workflow.jpg	660.7 KB	Public marketing workflow graphic
5	frontend/dist/hero_dashboard.jpg	512.0 KB	Public landing page hero graphic
6	frontend/dist/assets/xlsx-*.js	411.9 KB	Client-side Excel import/export engine chunk
7	frontend/dist/assets/index-*.js	396.3 KB	Main compiled frontend bundle entry point
8	frontend/dist/assets/jspdf.es.min-*.js	390.1 KB	Client-side PDF rendering engine chunk
9	frontend/dist/assets/Analytics-*.js	379.3 KB	Analytics dashboards & charts bundle chunk
10	frontend/dist/assets/html2canvas-*.js	194.9 KB	Canvas screenshot generation chunk

32. Dependency Inventory

Backend Dependencies (Production)

Package	Version	Purpose in Application
express	5.2.1	Core HTTP REST API web server framework
mongoose	9.7.1	MongoDB Object Document Mapper & connection pool
jsonwebtoken	9.0.3	Cryptographic JWT token signing & verification
bcryptjs	3.0.3	Password hashing with salt rounds
razorpay	2.9.8	Official Razorpay payment gateway Node.js SDK
cloudinary	2.x	Cloud media storage & CDN upload pipeline
multer	2.2.0	Multipart/form-data upload stream parser
streamifier	0.1.5	Streams in-memory file buffers to Cloudinary
node-cron	4.5.0	Background task scheduling (warranties, SLAs)
nodemailer	9.0.1	Transactional SMTP email dispatch (invites, alerts)
qrcode	1.5.4	Server-side QR code generator for asset tags
express-rate-limit	8.5.2	IP rate limiting on authentication routes
dotenv	17.4.2	Environment variable configuration management
cors	2.8.6	Cross-Origin Resource Sharing security headers

33. Environment Variables Specification

Variable Name	Classification	Purpose & Technical Requirement
PORT	Optional (Default: 5000)	HTTP listening port for the Express backend server
MONGO_URI	Required	MongoDB Atlas connection string with replica set & auth
JWT_SECRET	Required	Cryptographic secret key for signing user authentication tokens
NODE_ENV	Required	Runtime mode ('production' or 'development')
RAZORPAY_KEY_ID	Required (Billing)	Public merchant key ID provided by Razorpay dashboard
RAZORPAY_KEY_SECRET	Required (Billing)	Private API secret used for HMAC SHA256 signature verification
RAZORPAY_WEBHOOK_SECRET	Optional	HMAC secret for verifying server-to-server webhook callbacks
CLOUDINARY_CLOUD_NAME	Optional	Cloudinary cloud account identifier for CDN storage
CLOUDINARY_API_KEY	Optional	Public API key for Cloudinary REST authentication
CLOUDINARY_API_SECRET	Optional	Private secret for Cloudinary stream signing
SMTP_HOST / SMTP_PORT	Optional	Outgoing mail server hostname (e.g. smtp.gmail.com) and port (587)
SMTP_USER / SMTP_PASS	Optional	Authenticated mail account credentials for email alerts
FRONTEND_URL	Optional (Default: 5173)	Base URL for email invite links and password reset redirects
DESKTOP_APP_DOWNLOAD_URL	Optional	Cloud hosting link for downloading the standalone installer

34. Quality Assurance & Testing History

The system has undergone extensive automated and manual end-to-end testing across all core modules:

Test Domain	Tests Executed	Verified Results & Status
Duplicate Registration	Colliding slug & duplicate company name inputs	Passed: Returns clean HTTP 400 without stack traces
Monetary Rounding	Coupon discounts & GST calculations with odd cents	Passed: All fields rounded to exactly 2 decimal places
Super Admin Authorization	Company Admin attempting to access /api/super-admin/*	Passed: Returns HTTP 403 Forbidden with security audit log
Tenant Isolation	Tenant A attempting to query Tenant B's asset IDs	Passed: Returns null/404; cross-tenant leakage impossible
Cloudinary Pipeline	Upload SVG/PNG buffer to Cloudinary with deletion	Passed: Cloudinary status OK; secure HTTPS URLs generated
Encoding Verification	Full codebase scan for corrupted characters (mojibake)	Passed: 0 corrupted characters across all frontend files

36. Current Production Database Metrics

Metric	Live Count	Description
Registered Tenants	3 Tenants	Active company accounts (icpl , trai , kedia)
Registered Users	19 Users	Company Admins, HODs, Technicians, Employees
Subscription Records	54 Records	Lifecycle audit history across all tenant subscriptions
Active Coupons	1 Coupon	Configured promotional discount coupons
Invoices Generated	1 Invoice	Live commercial invoices recorded
API Keys Issued	2 Keys	External developer integration tokens

40. Overall System Status Matrix

Subsystem Area	Implementation Status	Operational Readiness
Core Asset Registry & QR	✔ IMPLEMENTED	Production Ready
Authentication & RBAC	✔ IMPLEMENTED	Production Ready
Multi-Tenant Data Enclosure	✔ IMPLEMENTED	Production Ready
Subscription Engine (365d)	✔ IMPLEMENTED	Production Ready
Coupon & Discount Engine	✔ IMPLEMENTED	Production Ready
GST Calculation (CGST/SGST/IGST)	✔ IMPLEMENTED	Production Ready
Razorpay Payment Verification	✔ IMPLEMENTED	Production Ready (Test/Live Switchable)
Cloud Storage (Cloudinary)	✔ IMPLEMENTED	Active & Verified Live
Super Admin Platform Console	✔ IMPLEMENTED	Production Ready
Company Deactivation Guard	✔ IMPLEMENTED	Production Ready
Windows Desktop Installer	✔ IMPLEMENTED	Inno Setup Installer Compiled
Scheduled Plan Downgrades	✘ NOT IMPLEMENTED	Immediate block active; scheduled in backlog

41. Management Summary

Executive Briefing for Leadership

AssetCare has reached complete core software maturity. The system successfully consolidates asset lifecycle tracking, maintenance scheduling, ticket escalation, and user management into a single, cohesive multi-tenant portal.

From a commercial standpoint, the platform features a complete subscription engine supporting 3 annual plan tiers with automated GST invoice generation, discount coupons, and Razorpay integration. Organizations are securely isolated in their own workspaces, and platform owners have full oversight via the Super Admin Console.

Remaining Actions Before Commercial Launch:

1. Switch Razorpay keys from Test Mode to Live Production Mode in the production `.env` file.
2. Configure production SMTP credentials for outgoing client email invitations and ticket alerts.
3. Attach custom corporate domain and SSL certificate on the production hosting environment.

42. Architectural Conclusion

The AssetCare architecture delivers a balance of developer velocity, enterprise security, and operational reliability:

- **Modular Decoupling:** Frontend and backend communicate strictly over authenticated RESTful contracts, allowing independent scaling and headless operations.
- **Bulletproof Isolation:** Multi-tenant design prevents cross-customer leakage at both the query level and middleware level.
- **Server Authoritative Billing:** All financial and entitlement logic is computed server-side, eliminating financial tampering risks.
- **Deployment Flexibility:** The codebase seamlessly compiles into both cloud-hosted SaaS infrastructure and zero-config Windows desktop packages.

Appendix A: Master Route & API Inventory

HTTP Method	Endpoint Route	Role Requirement	Tenant Scope	Description
POST	/api/auth/register-company	Public	Global	Register new company & create tenant admin
POST	/api/auth/login	Public	Global	Authenticate user, verify tenant, issue JWT
GET	/api/auth/me	Authenticated	Tenant Scoped	Retrieve fresh user profile & permissions
GET/POST	/api/assets	Admin / Staff	Tenant Scoped	List assets (with search/filters) or create asset
POST	/api/assets/bulk-import	Admin	Tenant Scoped	Bulk import assets from CSV file
GET/POST	/api/tickets	All Roles	Tenant Scoped	List or raise breakdown service tickets
POST	/api/billing/create-order	Admin	Tenant Scoped	Calculate GST/coupons & create Razorpay order
POST	/api/billing/verify-payment	Admin	Tenant Scoped	Verify HMAC signature & activate 365d license
POST	/api/billing/validate-coupon	Admin	Global	Validate promo code and return discount amount
GET	/api/super-admin/overview	Super Admin	Global	Platform overview metrics & revenue statistics
PUT	/api/super-admin/tenants/:id/status	Super Admin	Global	Freeze/deactivate or reactivate tenant
POST	/api/super-admin/coupons	Super Admin	Global	Create new promotional discount coupon

Appendix B: Database Collections & Schemas

Collection	Primary Key	Unique Indices	Relational References
tenants	_id	slug: 1	None (Master entity)
users	_id	email: 1	tenantId → tenants.slug
assets	_id	assetTag: 1 , serialNumber: 1	assignedTo → users._id , department → departments._id
tickets	_id	ticketId: 1	asset → assets._id , raisedBy → users._id
invoices	_id	invoiceNumber: 1	tenantId → tenants.slug , uploadedBy → users._id
coupons	_id	code: 1	None (Master entity)
subscriptionhistories	_id	None	tenantId → tenants.slug